Kadane's Algorithm
Maximum Sum Subarray

Contents

1 Problem Statement

Given an integer array

$$A = [a_0, a_1, \ldots, a_{n-1}]$$

find the contiguous subarray having the maximum possible sum.
Example

$$[-2, 1, -3, 4, -1, 2, 1, -5, 4]$$

Maximum Sum:

$$4 + (-1) + 2 + 1 = 6$$

Maximum Sum Subarray:

$$[4, -1, 2, 1]$$

2 Observations

The keyword is **contiguous**.
We cannot choose arbitrary elements.
For every index, we need to decide

- Should we continue the previous subarray?
- Or should we start a brand new subarray from the current element?

This single decision leads to Kadane's Algorithm.

3 Brute Force Approach

Generate every possible subarray.
For every subarray,

- Compute its sum.
- Update the maximum.

Number of subarrays:

$$\frac{n(n+1)}{2}$$

Time complexity:

$$O(n^3)$$

or

$$O(n^2)$$

using prefix sums.
Still too slow for large inputs.

4 Key Insight

Suppose we are currently at index i.
We already know the best subarray ending at index $i-1$.
Should we include it?
There are only two possibilities.

Case 1

Continue the previous subarray.
Current sum becomes

$$\text{currentSum} + A[i]$$

Case 2

Start a new subarray from the current element.
Current sum becomes

$$A[i]$$

Therefore,

$$\boxed{currentSum = \max(A[i], currentSum + A[i])}$$

This equation is the heart of Kadane's Algorithm.

5 Why Does This Work?

Suppose

$$currentSum = -10$$

Current element

$$A[i] = 5$$

If we extend,

$$-10 + 5 = -5$$

If we start fresh,

$$5$$

Clearly,
starting a new subarray is better.

Now suppose

$$currentSum = 8$$

Current element

$$A[i] = 5$$

Extending gives

$$8 + 5 = 13$$

Starting new gives

$$5$$

Clearly extending is better.
Thus at every step,
we simply choose whichever is larger.

6 Algorithm

Maintain two variables.

1. currentSum
2. maxSum

Initially,

$$currentSum = A[0]$$

$$maxSum = A[0]$$

For every remaining element,

$$currentSum = \max(A[i], currentSum + A[i])$$

Then,

$$maxSum = \max(maxSum, currentSum)$$

7 Dry Run

Array

$$[-2, 1, -3, 4, -1, 2, 1, -5, 4]$$

Index	Element	Current Sum	Max Sum
0	-2	-2	-2
1	1	1	1
2	-3	-2	1
3	4	4	4
4	-1	3	4
5	2	5	5
6	1	6	6
7	-5	1	6
8	4	5	6

Answer

$$6$$

8 Visual Intuition

Suppose

$$4, -1, 2, 1$$

Running sums

$$4$$

$$4 + (-1) = 3$$

$$3 + 2 = 5$$

$$5 + 1 = 6$$

The running sum always stays positive.
Therefore extending the subarray is beneficial.

Now suppose

$$4, -8$$

Running sum becomes

$$-4$$

A negative running sum hurts every future subarray.
Hence we discard it completely.

9 Proof of Correctness

Assume we are processing index i.
The best subarray ending at i must be one of the following:

1. The element itself.
2. Previous best ending at $i - 1$ extended by $A[i]$.

No third possibility exists.
Hence,

$$currentSum = \max(A[i], currentSum + A[i])$$

correctly computes the best subarray ending at index i.
Keeping the maximum over all indices gives the global optimum.
Thus Kadane's Algorithm is correct.

10 Handling All Negative Numbers

Many beginners initialize

$$currentSum = 0$$

This is incorrect.
Example

$$[-5, -2, -8]$$

Correct answer

$$-2$$

Incorrect implementation returns

$$0$$

Therefore initialize with the first element.

Always initialize

$$currentSum = A[0]$$

and

$$maxSum = A[0]$$

instead of zero.

11 C++ Implementation

```cpp
int maxSubArray(vector<int>& nums) {

    int currentSum = nums[0];
    int maxSum = nums[0];

    for(int i = 1; i < nums.size(); i++) {

        currentSum = max(nums[i],
                         currentSum + nums[i]);

        maxSum = max(maxSum, currentSum);
    }

    return maxSum;
}
```

12 Returning the Actual Subarray

Maintain three indices.

- tempStart

- start
- end

Whenever we restart,

$$tempStart = i$$

Whenever a new maximum is found,

$$start = tempStart$$

$$end = i$$

The subarray is

$$[start, end]$$

13 Time Complexity

Each element is processed exactly once.

$$\boxed{O(n)}$$

14 Space Complexity

Only two variables are maintained.

$$\boxed{O(1)}$$

15 When Should You Think of Kadane's Algorithm?

Look for the following clues.

- Maximum subarray
- Minimum subarray
- Maximum contiguous segment
- Largest profit over consecutive days
- Maximum difference using continuous interval
- Circular subarray problems
- Dynamic Programming with local optimum

16 Common Mistakes

1. Initializing answer with zero.
2. Forgetting all-negative arrays.
3. Applying Kadane to non-contiguous problems.
4. Updating maxSum before currentSum.
5. Using nested loops unnecessarily.

17 Summary

At every element, ask only one question:

$$\boxed{\text{Should I extend the previous subarray, or should I start a new one?}}$$

Mathematically,

$$\boxed{currentSum = \max(A[i], currentSum + A[i])}$$

Update the global answer using

$$\boxed{maxSum = \max(maxSum, currentSum)}$$

This simple recurrence gives an optimal

$$\boxed{O(n)}$$

solution with

$$\boxed{O(1)}$$

extra space.